

innovate

achieve

lead



**BITS Pilani**

Pilani Campus

# Natural Language Processing

Dr. Chetana Gavankar, Ph.D,  
IIT Bombay-Monash University Australia  
[Chetana.gavankar@pilani.bits-pilani.ac.in](mailto:Chetana.gavankar@pilani.bits-pilani.ac.in)



Grateful Acknowledgment: Source of this slide deck

**Jurafsky and Martin:** <https://web.stanford.edu/~jurafsky/slp3/>  
[https://web.stanford.edu/~jurafsky/slp3/slides/7\\_NN\\_Apr\\_28\\_2021.pptx](https://web.stanford.edu/~jurafsky/slp3/slides/7_NN_Apr_28_2021.pptx)

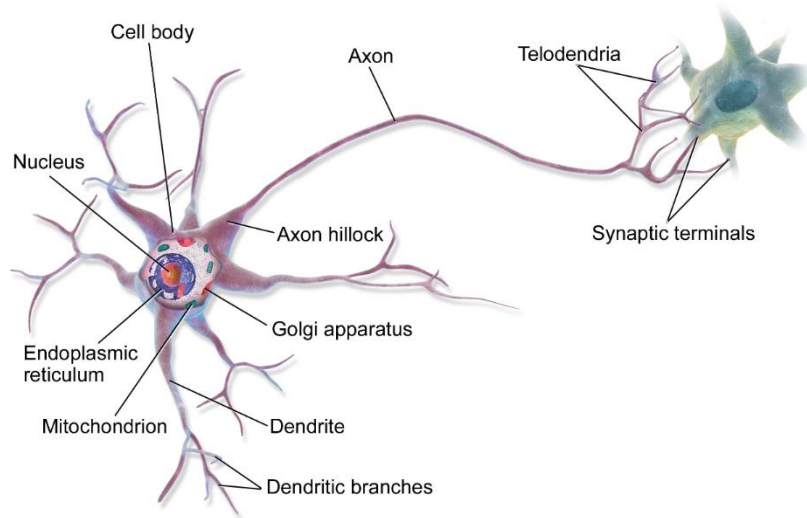
# Session Content

---

## Neural Networks and Neural Language Models

- Units in Neural Networks
- The XOR problem
- Feedforward Neural Networks
- Applying Feedforward Neural Networks to NLP tasks
- Training Neural Networks: Overview
- Neural Language Models
- Introduction to large and small language models (LLM and SLM)
- Prompt Engineering

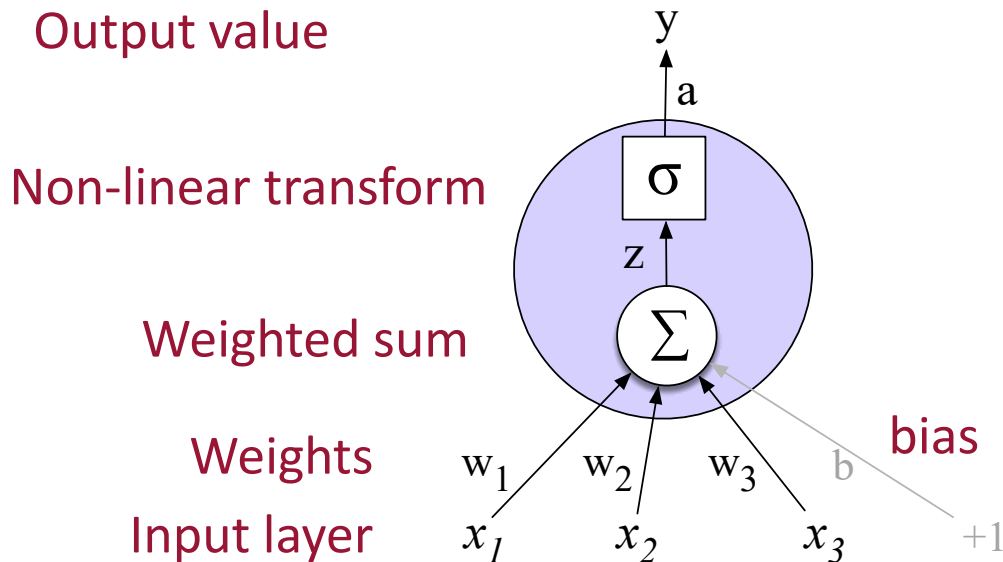
# This is in your brain



By BruceBlaus - Own work, CC BY 3.0,  
<https://commons.wikimedia.org/w/index.php?curid=28761830>

# Neural Network Unit

This is not in your brain



# Neural unit



- Take weighted sum of inputs, plus a bias

$$z = b + \sum_i w_i x_i$$

$$z = w \cdot x + b$$

- Instead of just using  $z$ , we'll apply a nonlinear activation function  $f$ :

$$y = a = f(z)$$

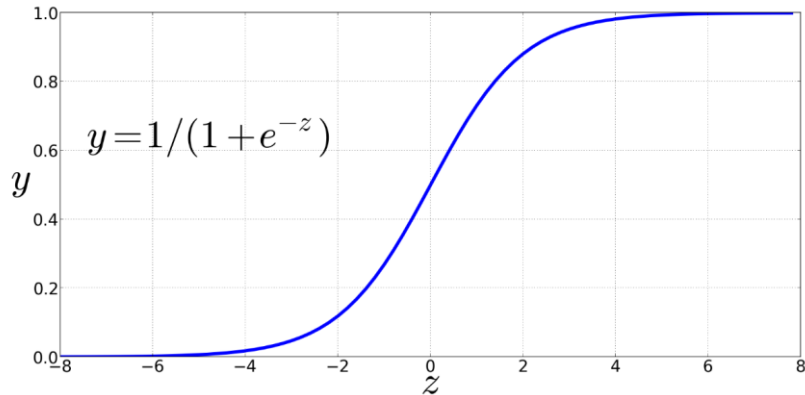
# Non-Linear Activation Functions



We've already seen the sigmoid for logistic regression:

**Sigmoid**

$$y = s(z) = \frac{1}{1 + e^{-z}}$$

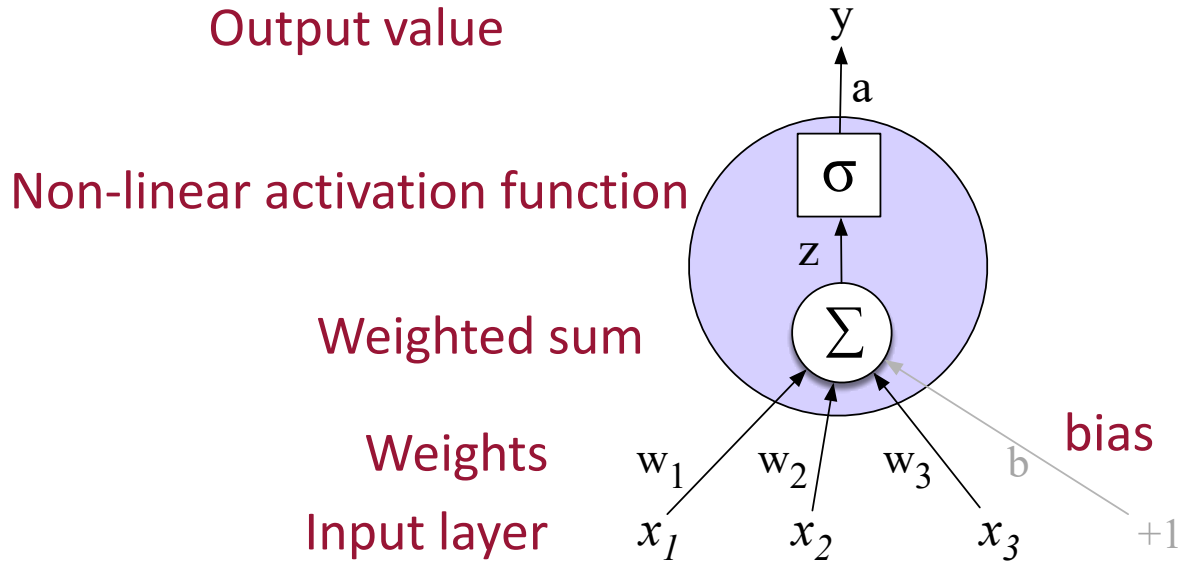


# Final function the unit is computing



$$y = s(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

# Final unit again



# An example



- *Suppose a unit has:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- What happens with input  $x$ :
- $x = [0.5, 0.6, 0.1]$

$$y = s(w \cdot x + b) =$$

# An example



- *Suppose a unit has:*
  - $w = [0.2, 0.3, 0.9]$
  - $b = 0.5$
- What happens with the following input  $x$ ?
- $x = [0.5, 0.6, 0.1]$

$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} =$$

# An example



- *Suppose a unit has:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- What happens with input  $x$ :
- $x = [0.5, 0.6, 0.1]$

$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} = \frac{1}{1 + e^{-(.5 \cdot .2 + .6 \cdot .3 + .1 \cdot .9 + .5)}} =$$

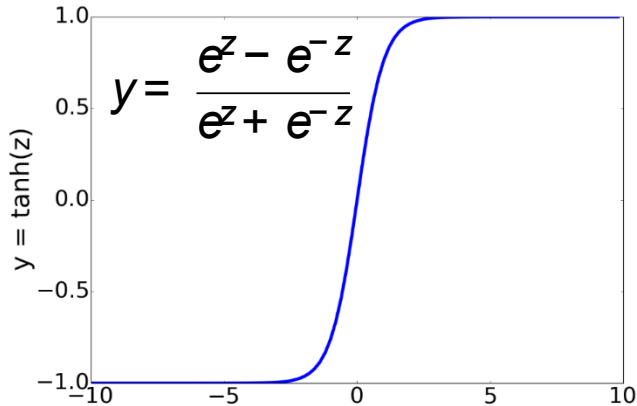
# An example



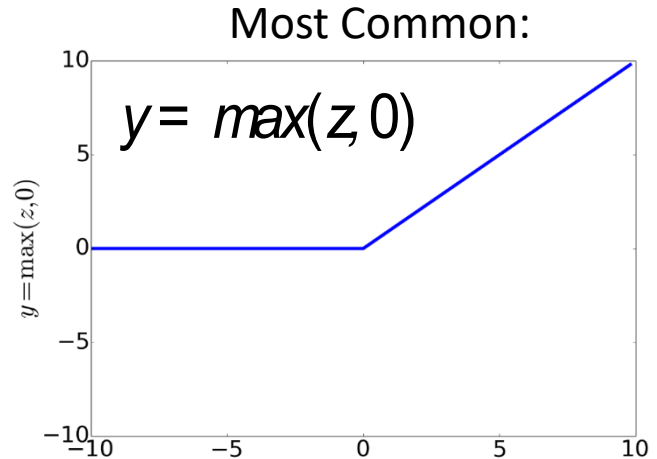
- *Suppose a unit has:*
- $w = [0.2, 0.3, 0.9]$
- $b = 0.5$
- What happens with input  $x$ :
- $x = [0.5, 0.6, 0.1]$

$$y = s(w \cdot x + b) = \frac{1}{1 + e^{-(w \cdot x + b)}} = \frac{1}{1 + e^{-0.87}} = .70$$

# Non-Linear Activation Functions besides sigmoid



tanh



ReLU  
Rectified Linear Unit

# Nonlinear function: softmax

- For a vector  $z$  of dimensionality  $k$ , the softmax is:

$$\text{softmax}(z) = \left[ \frac{\exp(z_1)}{\sum_{i=1}^k \exp(z_i)}, \frac{\exp(z_2)}{\sum_{i=1}^k \exp(z_i)}, \dots, \frac{\exp(z_k)}{\sum_{i=1}^k \exp(z_i)} \right]$$

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^k \exp(z_j)} \quad 1 \leq i \leq k$$

- Example:

$$z = [0.6, 1.1, -1.5, 1.2, 3.2, -1.1]$$

$$\text{softmax}(z) = [0.055, 0.090, 0.006, 0.099, 0.74, 0.010]$$

# Advanced nonlinear functions used in LLM: based on sigmoid



- **SiLU (Sigmoid Linear Unit) / Swish**
- **Usage:** Served as a stepping stone to SwiGLU and a standard activation in many architectures.
- **How it works:** Self-gated activation that allows smooth gradient flow
- **GELU (Gaussian Error Linear Unit)**
- **Usage:** Found in the GPT series (GPT-2, GPT-3), BERT, and early generations of open-weight models.
- **How it works:** Weights inputs by their statistical percentile, blending the properties of a standard dropout and ReLU. It behaves like a smooth, non-monotonic curve that allows a small, non-zero gradient for negative inputs.

# The XOR problem



Minsky and Papert (1969)

- Can neural units compute simple functions of input?

| AND |    |   | OR |    |   | XOR |    |   |
|-----|----|---|----|----|---|-----|----|---|
| x1  | x2 | y | x1 | x2 | y | x1  | x2 | y |
| 0   | 0  | 0 | 0  | 0  | 0 | 0   | 0  | 0 |
| 0   | 1  | 0 | 0  | 1  | 1 | 0   | 1  | 1 |
| 1   | 0  | 0 | 1  | 0  | 1 | 1   | 0  | 1 |
| 1   | 1  | 1 | 1  | 1  | 1 | 1   | 1  | 0 |

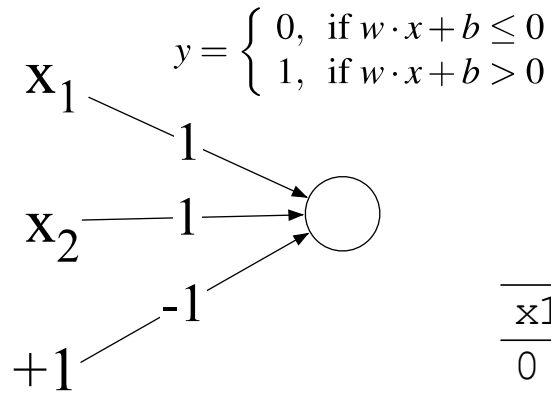
# Perceptrons



- A very simple neural unit
- Binary output (0 or 1)
- No non-linear activation function

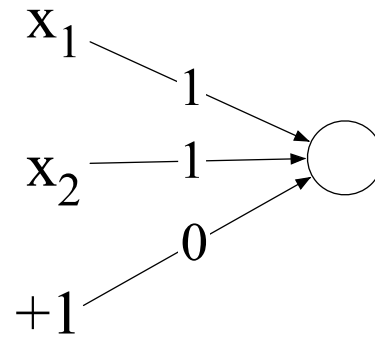
$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

# Easy to build AND or OR with perceptrons



**AND**

| AND |    |   |
|-----|----|---|
| x1  | x2 | y |
| 0   | 0  | 0 |
| 0   | 1  | 0 |
| 1   | 0  | 0 |
| 1   | 1  | 1 |



**OR**

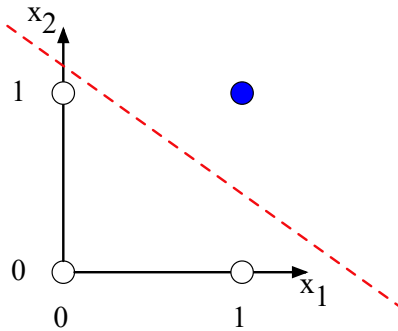
| OR |    |   |
|----|----|---|
| x1 | x2 | y |
| 0  | 0  | 0 |
| 0  | 1  | 1 |
| 1  | 0  | 1 |
| 1  | 1  | 1 |

# Why? Perceptrons are linear classifiers

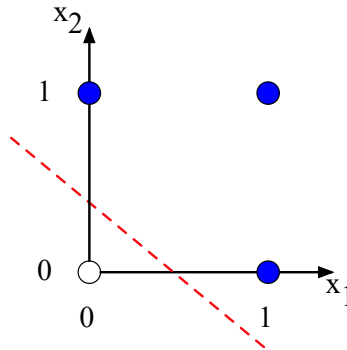


- Perceptron equation given  $x_1$  and  $x_2$ , is the equation of a line
- $w_1x_1 + w_2x_2 + b = 0$
- (in standard linear format:  $x_2 = (-w_1/w_2)x_1 + (-b/w_2)$  )
- This line acts as a **decision boundary**
  - 0 if input is on one side of the line
  - 1 if on the other side of the line

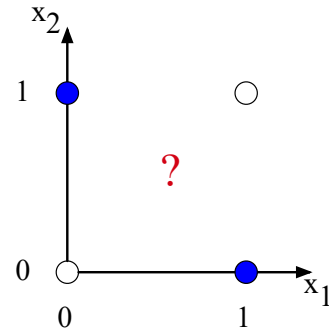
# Decision boundaries



a)  $x_1$  AND  $x_2$



b)  $x_1$  OR  $x_2$



c)  $x_1$  XOR  $x_2$

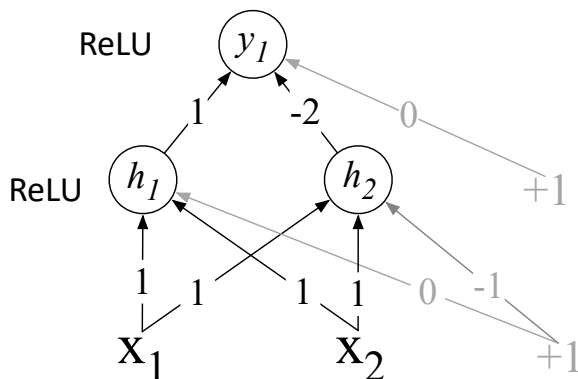
XOR is not a **linearly separable** function!

# Solution to the XOR problem



- XOR **can't** be calculated by a single perceptron
- XOR **can** be calculated by a layered network of units.

| XOR |    |   |
|-----|----|---|
| x1  | x2 | y |
| 0   | 0  | 0 |
| 0   | 1  | 1 |
| 1   | 0  | 1 |
| 1   | 1  | 0 |

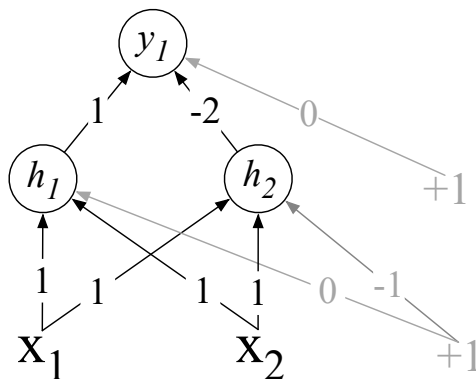


# Solution to the XOR problem

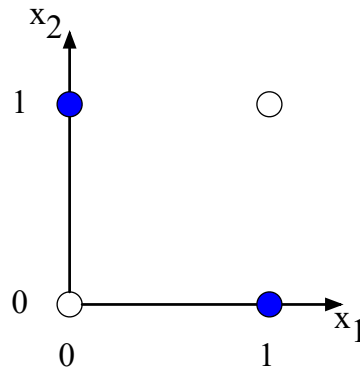


- XOR **can't** be calculated by a single perceptron
- XOR **can** be calculated by a layered network of units.

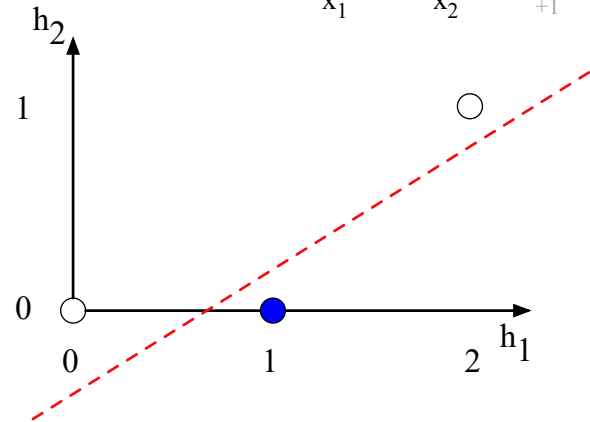
| XOR |    |   |
|-----|----|---|
| x1  | x2 | y |
| 0   | 0  | 0 |
| 0   | 1  | 1 |
| 1   | 0  | 1 |
| 1   | 1  | 0 |



# The hidden representation $h$

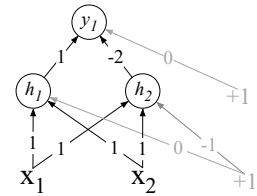


a) The original  $x$  space



b) The new (linearly separable)  $h$  space

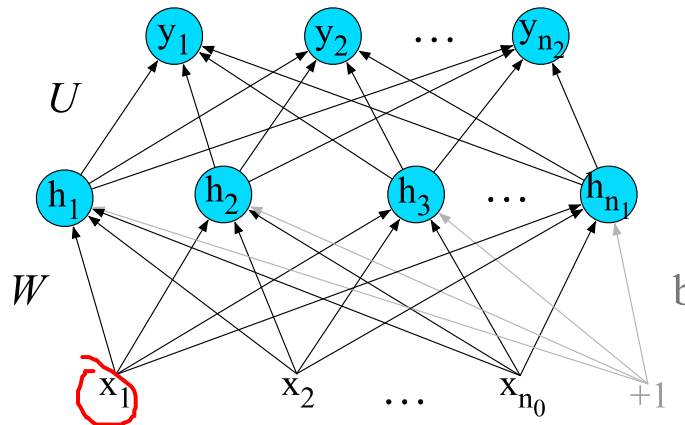
(With learning: hidden layers will learn to form useful representations)



# Feedforward Neural Networks

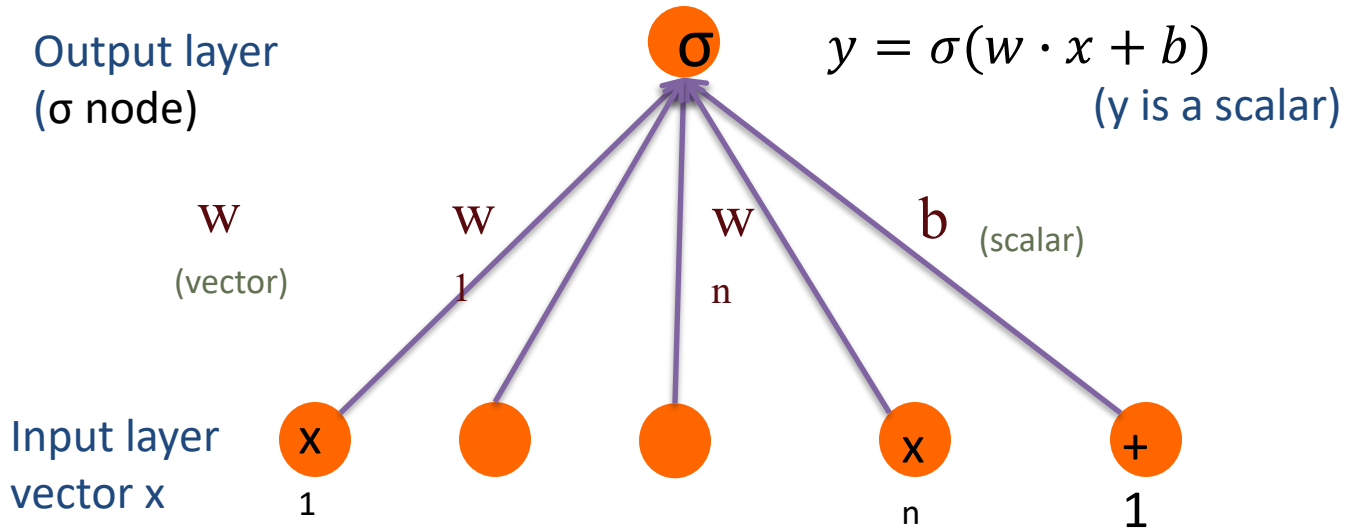


- Can also be called **multi-layer perceptrons** (or **MLPs**) for historical reasons

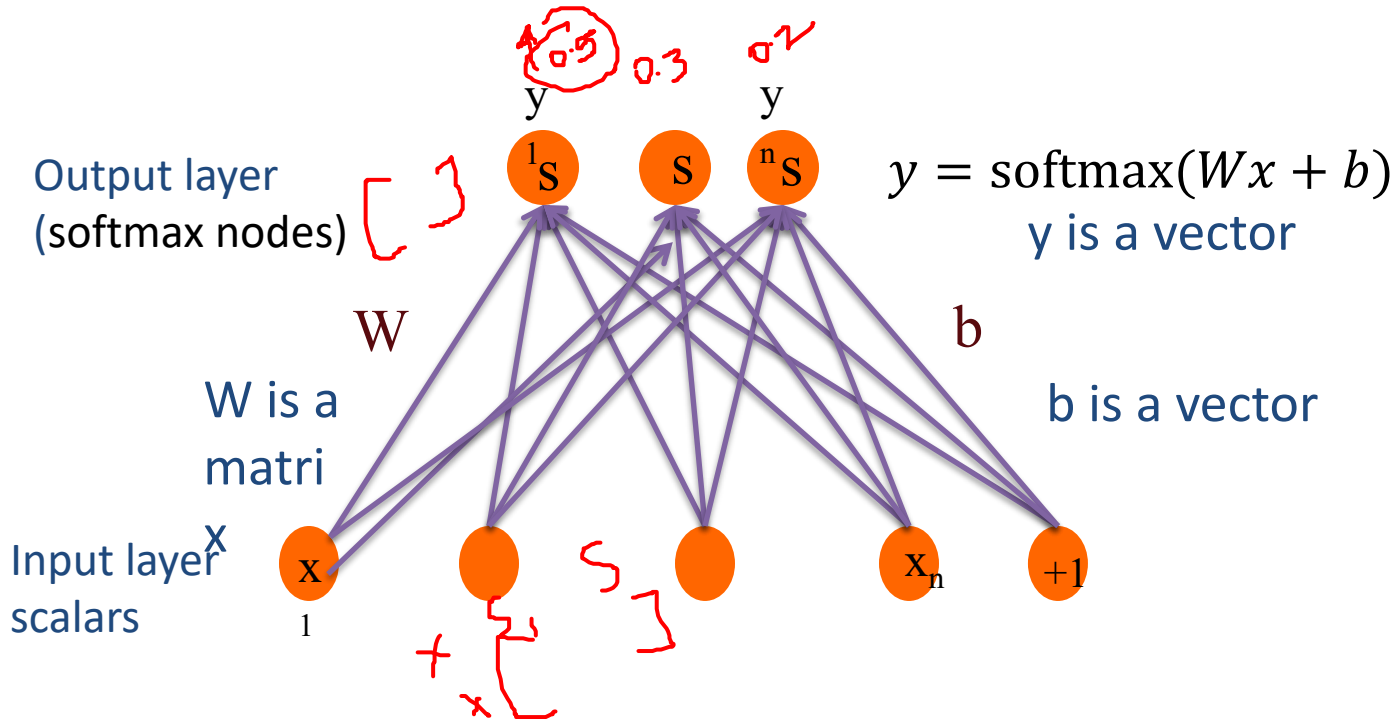


# Binary Logistic Regression as a 1-layer Network

(we don't count the input layer in counting layers!)



# Fully connected single layer network



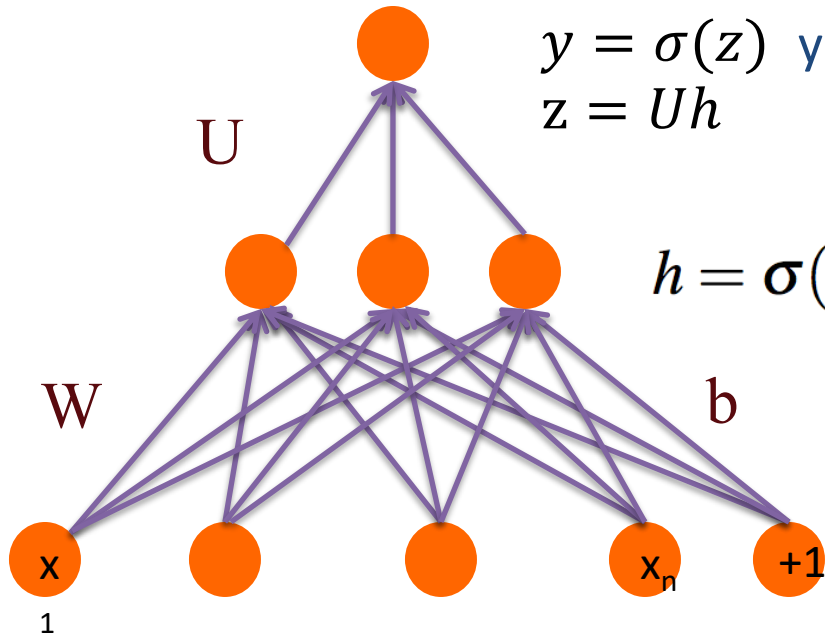
# Two-Layer Network with scalar output



Output layer  
( $\sigma$  node)

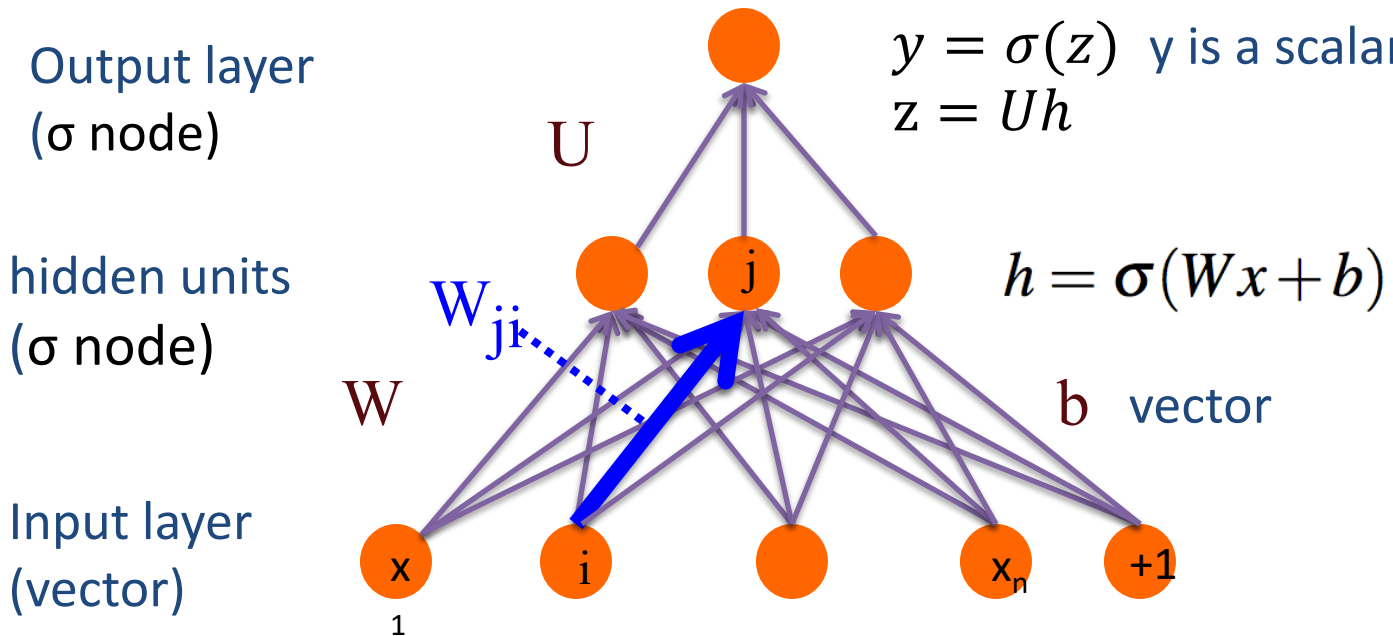
hidden units  
( $\sigma$  node)

Input layer  
(vector)



Could be ReLU  
Or tanh

# Two-Layer Network with scalar output



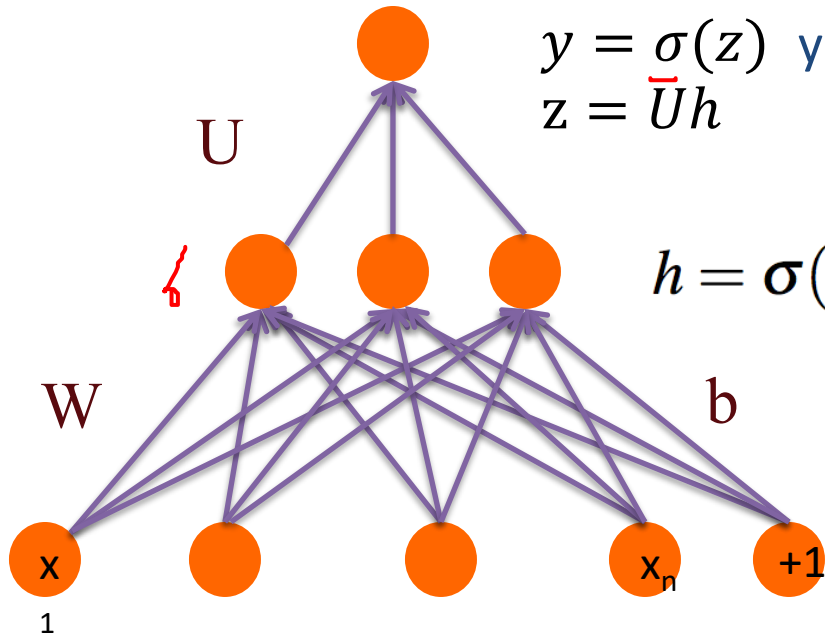
# Two-Layer Network with scalar output



Output layer  
( $\sigma$  node)

hidden units  
( $\sigma$  node)

Input layer  
(vector)



Could be ReLU  
Or tanh

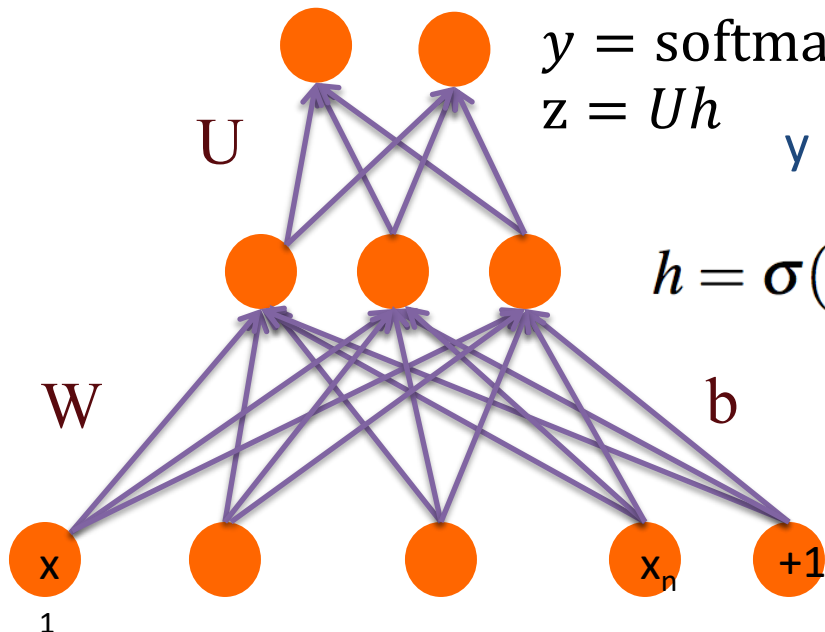
# Two-Layer Network with softmax output



Output layer  
( $\sigma$  node)

hidden units  
( $\sigma$  node)

Input layer  
(vector)

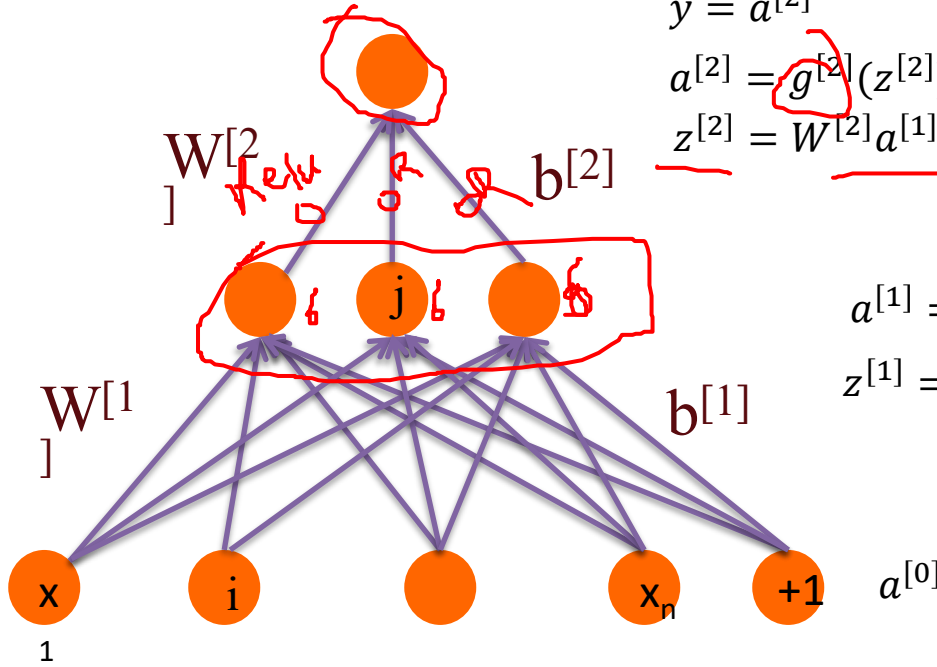


Could be ReLU  
Or tanh

# Multi-layer Notation



*BITT*  
*Inference*



$$y = a^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]}) \quad \text{sigmoid or softmax}$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[1]} = g^{[1]}(z^{[1]}) \quad \text{ReLU}$$

$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

# Multi Layer Notation



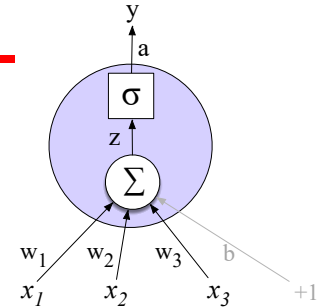
$$z^{[1]} = W^{[1]}a^{[0]} + b^{[1]}$$

$$a^{[1]} = g^{[1]}(z^{[1]})$$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

$$a^{[2]} = g^{[2]}(z^{[2]})$$

$$\hat{y} = a^{[2]}$$



**for  $i$  in 1..n**

$$z^{[i]} = W^{[i]} a^{[i-1]} + b^{[i]}$$

$$a^{[i]} = g^{[i]}(z^{[i]})$$

$$\hat{y} = a^{[n]}$$

# Replacing the bias unit



- Let's switch to a notation without the bias unit
- Just a notational change
  1. Add a dummy node  $a_0=1$  to each layer
  2. Its weight  $w_0$  will be the bias
  3. So input layer  $a^{[0]}_0=1$ ,
    - And  $a^{[1]}_0=1$  ,  $a^{[2]}_0=1, \dots$

# Replacing the bias unit



• Instead of:

$$x = x_1, x_2, \dots, x_{n0}$$

$$h = \sigma(Wx + b)$$

$$h_j = \sigma \left( \sum_{i=1}^{n_0} W_{ji} x_i + b_j \right)$$

We'll do this:

$$x = \underbrace{x_0}_{h_0}, x_1, x_2, \dots, x_{n0}$$

$$h = \sigma(Wx)$$

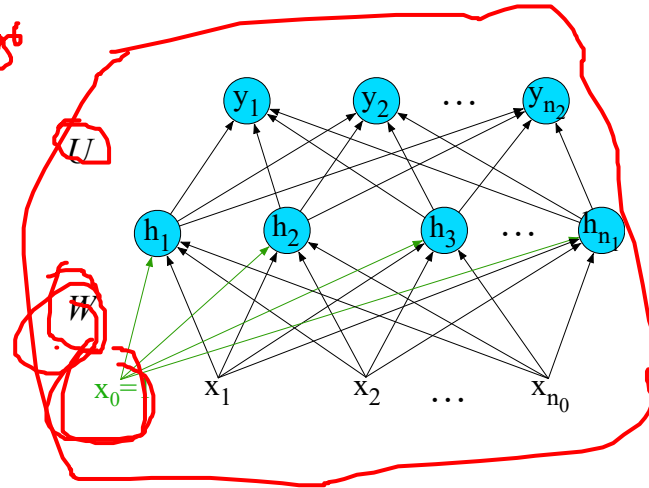
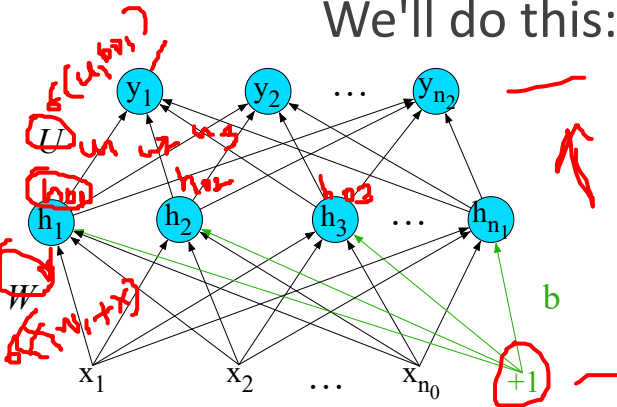
$$\sigma \left( \sum_{i=0}^{n_0} W_{ji} \underbrace{x_i}_{h_i} \right)$$

## Replacing the bias unit

## Instead of:

## We'll do this:

Few  
no one



# Use cases for feedforward networks

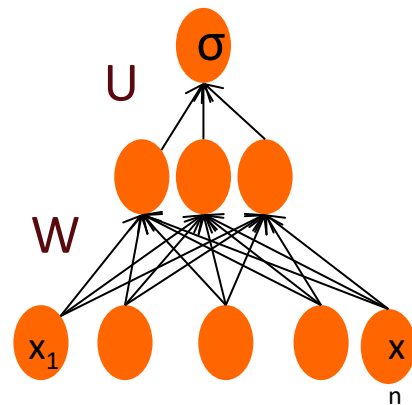


- Let's consider 2 (simplified) sample tasks:
  1. Text classification
  2. Language modeling
- State of the art systems use more powerful neural architectures, but simple models are useful to consider!

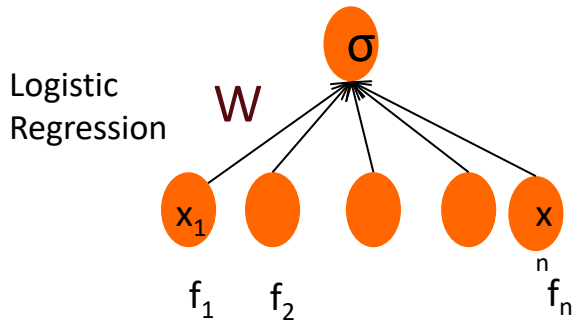
# Classification: Sentiment Analysis



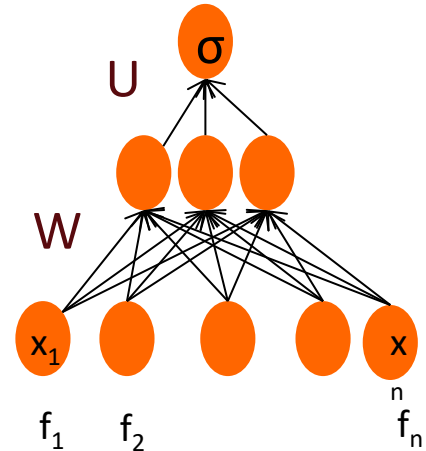
- We could do exactly what we did with logistic regression
- Input layer are binary features as before
- Output layer is 0 or 1



# Feedforward nets for simple classification



2-layer  
feedforward  
network

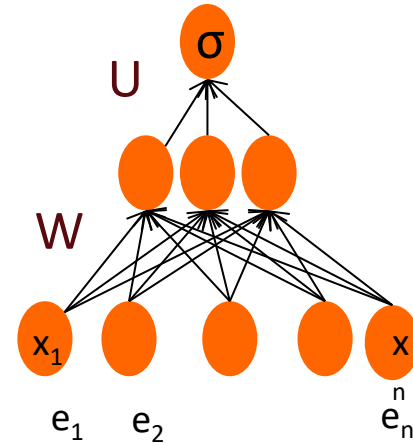


- Just adding a hidden layer to logistic regression
- allows the network to use non-linear interactions between features
- which may (or may not) improve performance.

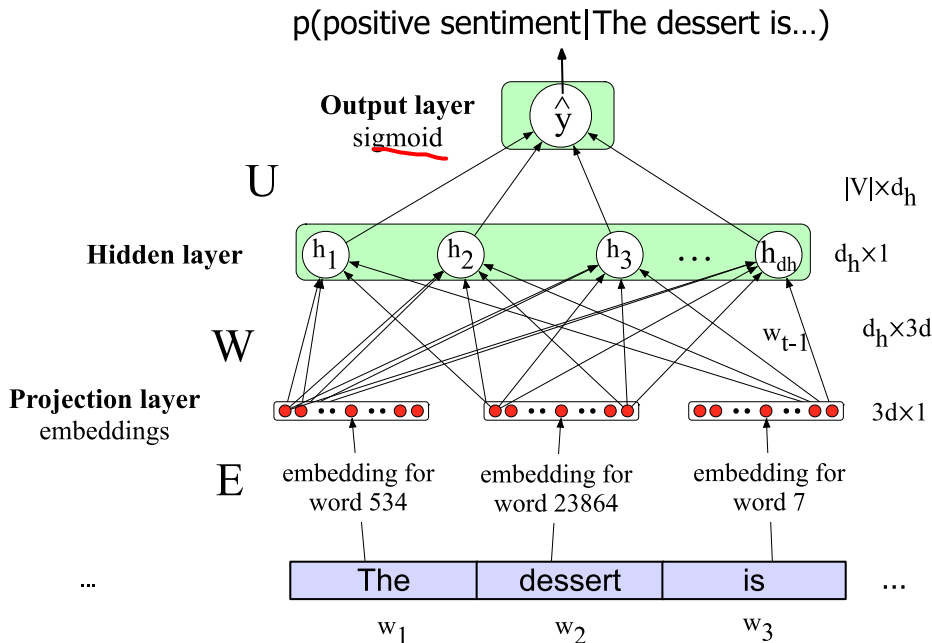
# Even better: representation learning



- The real power of deep learning comes from the ability to **learn** features from the data
- Instead of using hand-built human-engineered features for classification
- Use learned representations like embeddings!



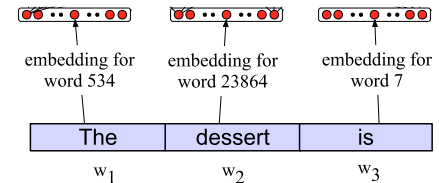
# Neural Net Classification with embeddings as input features!



# Issue: texts come in different sizes



- This assumes a fixed size length (3)!
- Kind of unrealistic.
- Some simple solutions (more sophisticated solutions later)



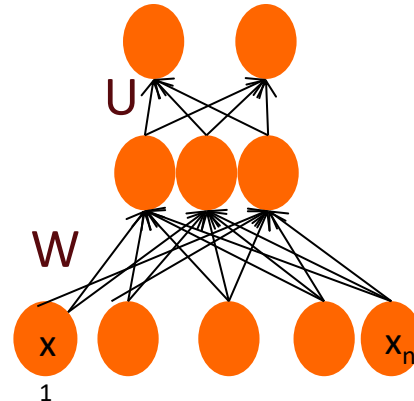
1. Make the input the length of the longest review
  - If shorter then pad with zero embeddings
  - Truncate if you get longer reviews at test time
2. Create a single "sentence embedding" (the same dimensionality as a word) to represent all the words
  - Take the mean of all the word embeddings
  - Take the element-wise max of all the word embeddings
    - For each dimension, pick the max value from all words

# Reminder: Multiclass Outputs



- What if you have more than two output classes?
  - Add more output units (one for each class)
  - And use a “softmax layer”

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^k e^{z_j}} \quad 1 \leq i \leq D$$



# Neural Language Models (LMs)



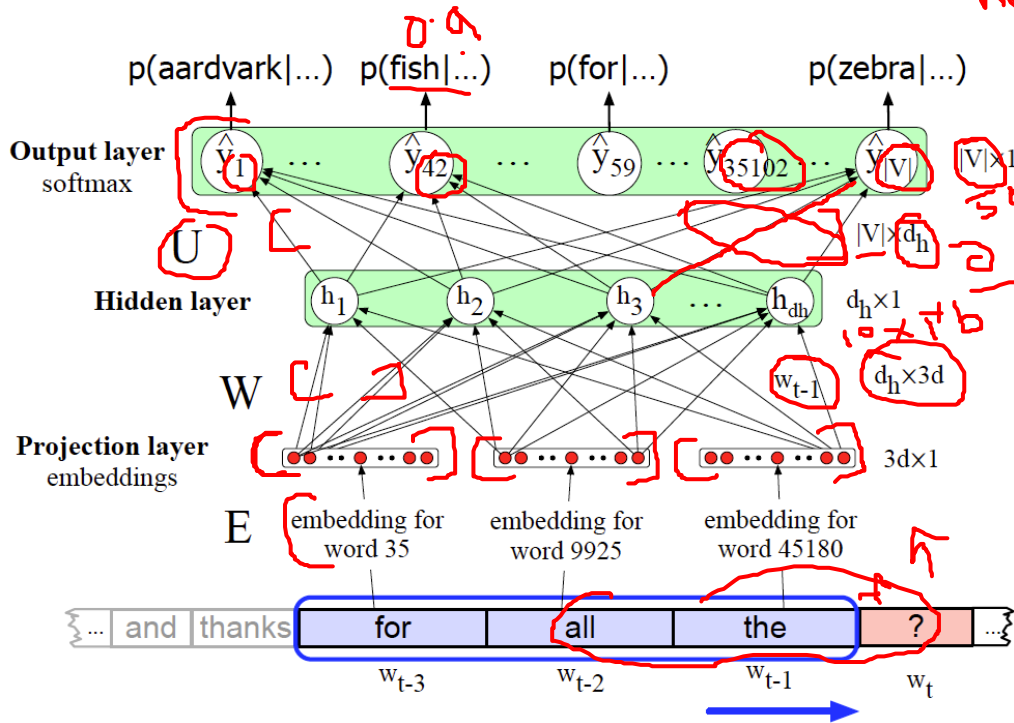
- **Language Modeling:** Calculating the probability of the next word in a sequence given some history.
- We've seen N-gram based LMs
- But neural network LMs far outperform n-gram language models
- State-of-the-art neural LMs are based on more powerful neural network technology like Transformers
- But **simple feedforward LMs** can do almost as well!

# Simple feedforward Neural Language Models

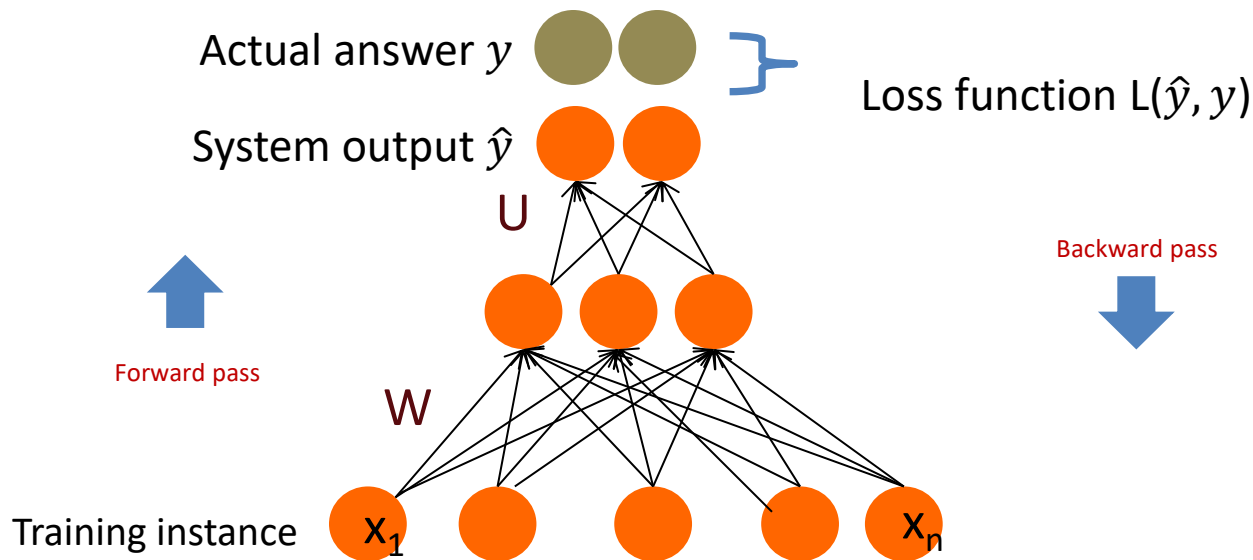
- **Task:** predict next word  $w_t$
- given prior words  $w_{t-1}, w_{t-2}, w_{t-3}, \dots$
- **Problem:** Now we're dealing with sequences of arbitrary length.
- **Solution:** Sliding windows (of fixed length)

$$P(w_t | w_1^{t-1}) \approx P(w_t | w_{t-N+1}^{t-1})$$

# Neural Language Model



# Intuition: training a 2-layer Network



# Intuition: Training a 2-layer network



- For every training tuple  $(x, y)$ 
  - Run *forward* computation to find our estimate  $\hat{y}$
  - Run *backward* computation to update weights:
    - For every output node
      - Compute loss  $L$  between true  $y$  and the estimated  $\hat{y}$
      - For every weight  $w$  from hidden layer to the output layer
        - » Update the weight
    - For every hidden node
      - Assess how much blame it deserves for the current answer
      - For every weight  $w$  from input layer to the hidden layer
        - » Update the weight

# Reminder: Loss Function for binary logistic regression



- A measure for how far off the current answer is to the right answer
- Cross entropy loss for logistic regression:

$$\begin{aligned} L_{\text{CE}}(\hat{y}, y) &= -\log p(y|x) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})] \\ &= -[y \log \sigma(w \cdot x + b) + (1 - y) \log(1 - \sigma(w \cdot x + b))] \end{aligned}$$

# Reminder: gradient descent for weight updates



- Use the derivative of the loss function with respect to weights  $\frac{d}{dw} L(f(x; w), y)$
- To tell us how to adjust weights for each training item
  - Move them in the opposite direction of the gradient

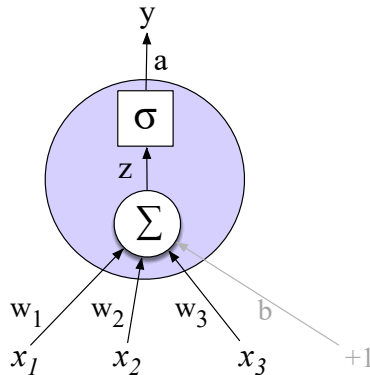
$$w^{t+1} = w^t - h \frac{d}{dw} L(f(x; w), y)$$

– For logistic regression

$$\frac{\partial L_{\text{CE}}(\hat{y}, y)}{\partial w_j} = [\sigma(w \cdot x + b) - y] x_j$$

# Where did that derivative come from?

- Using the chain rule!  $f(x) = u(v(x))$   $\frac{df}{dx} = \frac{du}{dv} \cdot \frac{dv}{dx}$
- Intuition (see the text for details)



Derivative of the weighted sum

Derivative of the Activation

Derivative of the Loss

$$\frac{\partial L}{\partial w_i} = \frac{\partial L}{\partial y} \frac{\partial y}{\partial z} \frac{\partial z}{\partial w_i}$$

# How can I find that gradient for every weight in the network?

---

- These derivatives on the prior slide only give the updates for one weight layer: the last one!
- What about deeper networks?
- Lots of layers, different activation functions?
- Solution in the next lecture:
- Even more use of the chain rule!!
- Computation graphs and backward differentiation!

# Neural Language Models (LMs)



- **Language Modeling:** Calculating the probability of the next word in a sequence given some history.
- We've seen N-gram based LMs
- But neural network LMs far outperform n-gram language models
- State-of-the-art neural LMs are based on more powerful neural network technology like **Transformers**
- But **simple feedforward LMs** can do almost as well!

# Simple feedforward Neural Language Models



- **Task:** predict next word  $w_t$   
given prior words  $w_{t-1}, w_{t-2}, w_{t-3}, \dots$
- **Output :** probability distribution over possible next words.

$$P(w_t | w_1, \dots, w_{t-1}) \approx P(w_t | w_{t-N+1}, \dots, w_{t-1})$$

- **Problem:** Now we're dealing with sequences of arbitrary length.
- **Solution:** Sliding windows (of fixed length)

# Word Embeddings

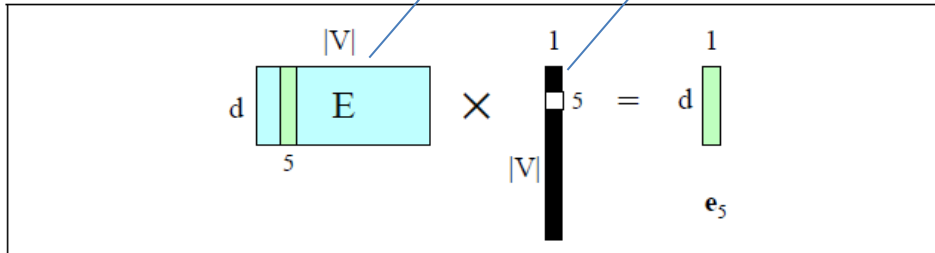


- one-hot vector representation , e.g., dog = (0,0,0,0,1,0,0,0,...), cat = (0,0,0,0,0,0,1,0,...)
- Represent each of the N previous words as a one-hot vector of length  $|V|$  one-hot vector , i.e., with one dimension for each word in the vocabulary
- word “toothpaste”, supposing it is  $V_5$ , i.e., index 5 in the vocabulary,  $x_5 = 1$ , and  $x_i = 0 \text{ } i \neq 5$ ,

$[0 \ 0 \ 0 \ 0 \ 1 \ 0 \ 0 \ \dots \ 0 \ 0 \ 0 \ 0]$   
 $1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ \dots \ \dots \ |V|$

Embedding matrix

One-hot vector



**Figure 7.12** Selecting the embedding vector for word  $V_5$  by multiplying the embedding matrix  $E$  with a one-hot vector with a 1 in index 5.

# Why Neural LMs work better than N-gram LMs

## embeddings

- Neural language models represent words in this prior context by their embeddings, rather than just by their word identity as used in n-gram language models.
- Using embeddings allows neural language models to generalize better to unseen data.

### Training data:

- We've seen: I have to make sure that the cat gets fed.
- Never seen: dog gets fed

### Test data:

- I forgot to make sure that the dog gets \_\_\_\_
- N-gram LM can't predict "fed"!
- Neural LM can use similarity of "cat" and "dog" embeddings to generalize and predict "fed" after dog

# N-gram vs NLM



- Compared to n-gram models, NLM can
  - handle much longer histories
  - generalize better over contexts of similar words,
  - Are more accurate at word-prediction.
- NLM are
  - much more complex,
  - slower
  - need more energy to train,
  - less interpretable than n-gram models, so for many (especially smaller) tasks an n-gram language model is still the right tool.



# N-gram versus LLM(neural language models)

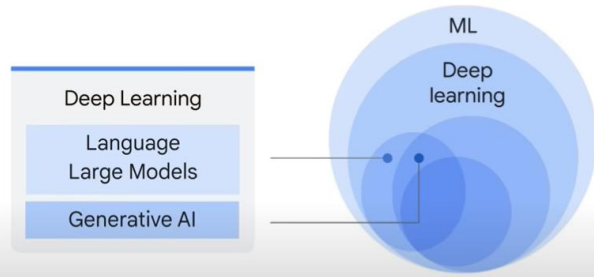
| Feature                      | N-gram Models                      | LLMs                  |
|------------------------------|------------------------------------|-----------------------|
| <b>Training Data Needed</b>  | Very small                         | Huge                  |
| <b>Compute</b>               | Low                                | Very high             |
| <b>Interpretability</b>      | Excellent                          | Poor                  |
| <b>Handles Long Context?</b> | No                                 | Yes                   |
| <b>Quality of Language?</b>  | Low-medium                         | Very high             |
| <b>Hallucination</b>         | Almost none                        | Possible              |
| <b>Deployment</b>            | Easy, lightweight                  | Heavy, expensive      |
| <b>Use Cases</b>             | Small, fast, interpretable systems | Large-scale NLP tasks |

# Large Language Models



Large, general-purpose language models can be pre-trained and then fine-tuned for specific purposes

Large Language Models (LLMs) also intersects with Generative AI

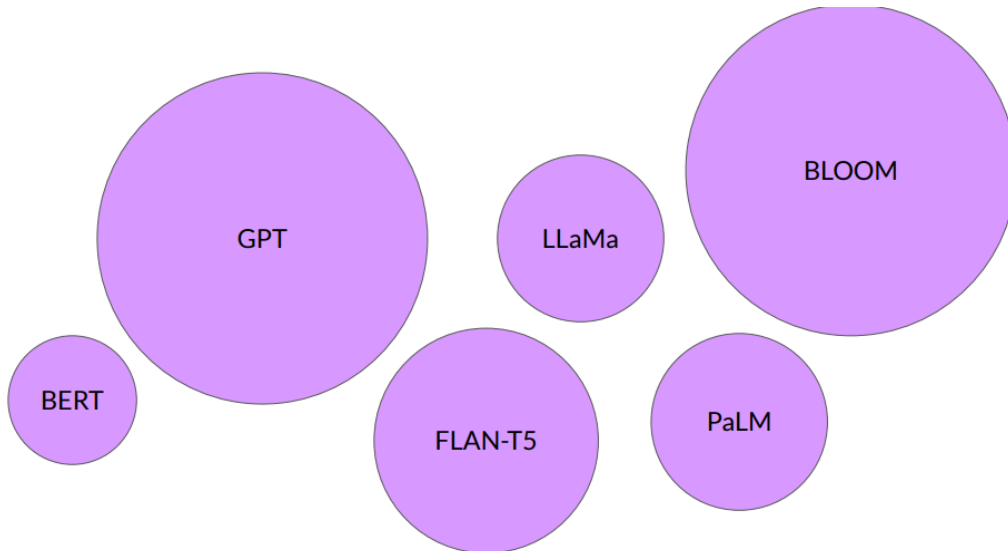


# Large Language Models

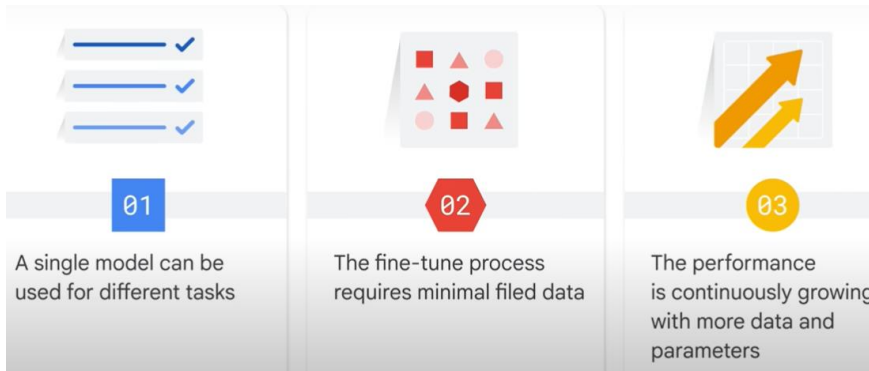
---

- ✓ Large
  - Large training dataset
  - Large number of parameters
- ✓ General purpose
  - Commonality of human languages
  - Resource restriction
- ✓ Pre-trained and fine-tuned

# Large Language Models



# Benefits of using Large Language Models



### LLM Development (using pre-trained APIs)

- NO ML expertise needed
- NO training examples
- NO need to train a model
- Thinks about prompt design

### Traditional ML Development

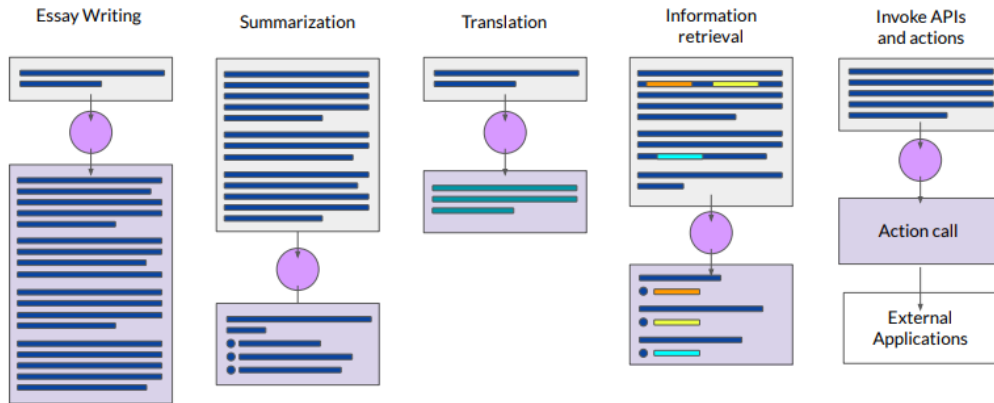
- YES ML expertise needed
- YES training examples
- YES need to train a model
- YES compute time +  
+ hardware
- Thinks about minimizing  
a loss function

# Fine Tuning



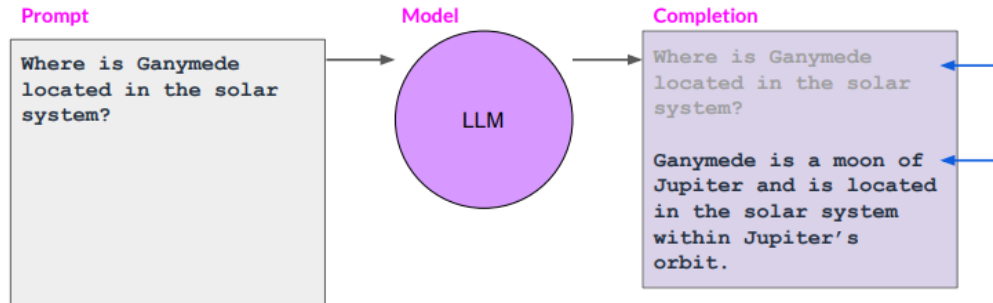
- Fine-tuning takes it a step further by allowing the pre-trained layers to be updated.
- Beneficial when the new dataset is large enough and similar to the original dataset on which the pre-trained model was trained

# LLM Use Cases



# Prompts and Prompt Engineering

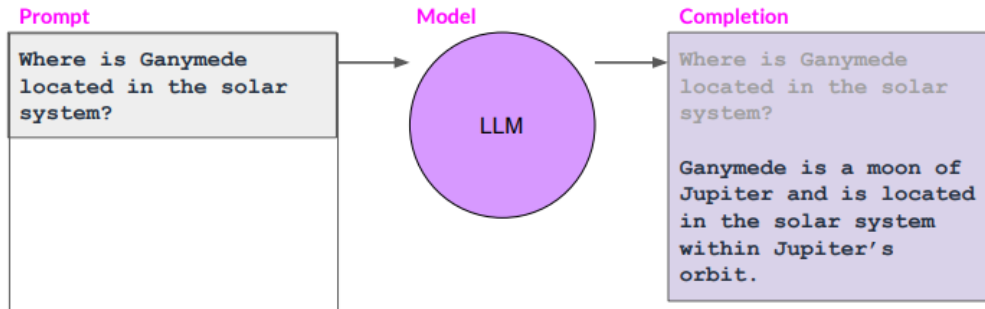
- Prompt engineering is the process of designing and refining the instructions (prompts) given to generative AI models to yield the most accurate, relevant, and useful responses.
- It acts as a bridge between human intent and machine understanding, ensuring AI outputs align with your specific goals



Context window

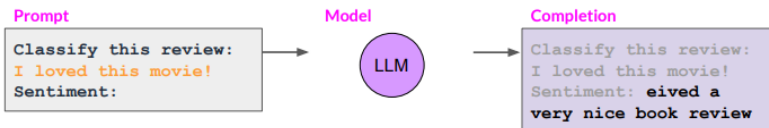
- typically a few 1000 words.

# Prompting and Prompt Engineering

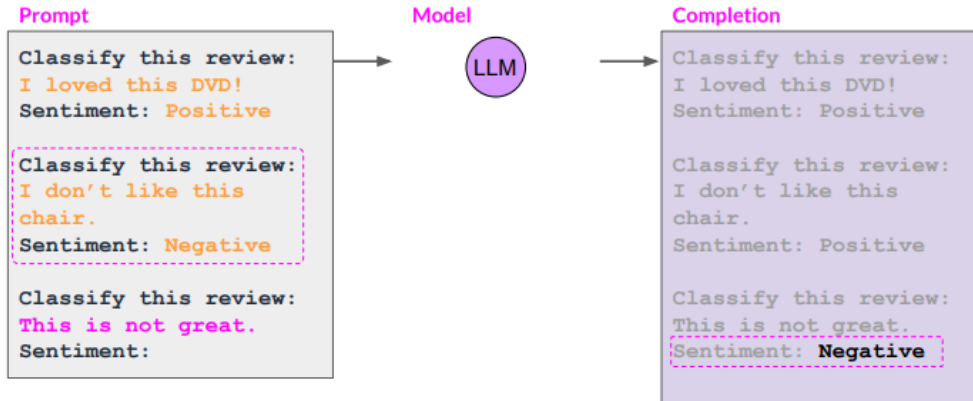


**Context window:** typically a few thousand words

# In context learning and zero shot inference



# In context learning and Few shot inference

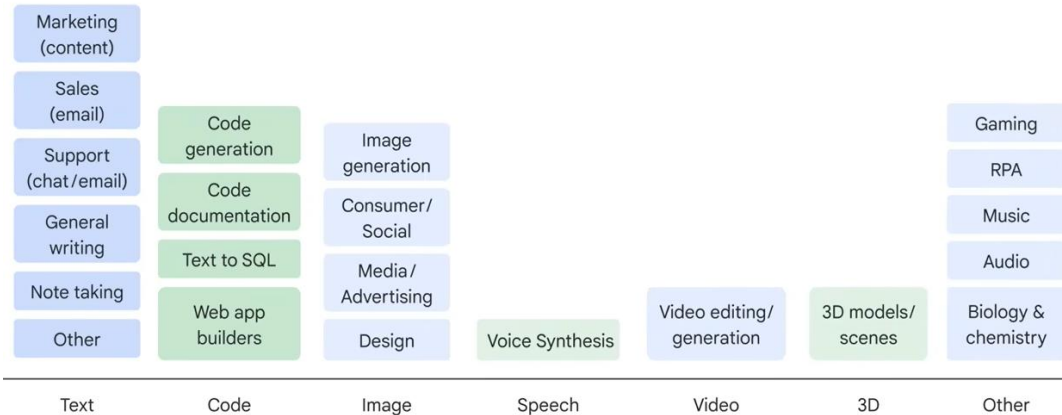


# Generative AI



- GenAI is a type of Artificial Intelligence that creates new content based on what it has learned from existing content.
- The process of learning from existing content is called training and results in the creation of a statistical model.
- When given a prompt, GenAI uses this statistical model to predict what an expected response might be—and this generates new content.

# Generative AI Applications



# Generative AI- AI Assistants

## Notification Assistant



Hi there - just a friendly reminder that your insurance policy expires in a month. Make sure to renew it in our member portal.

## FAQ Assistant

I need to renew my renters insurance. How much will it be?



You can calculate your renewal price on our website here: [xyz.com/renew](http://xyz.com/renew)

## Contextual Assistant

I need to renew my renters insurance. How much will it be?



I'd be happy to check for you. Firstly, are you still living in the same apartment?

Yes



Great - so just confirming it's 980 sq ft?

Yes



Thanks! Your new rate from September 1st onwards would be \$10 / month.



Would you like me to renew your policy for you right now?

Sure



Great. I've sent you a confirmation to your email.

# Generative AI- AI Assistants



## Personalized Assistant

- Assistant knows you much more in detail
- Quickly checks a few final things before giving you a quote tailored to your actual situation.



I can see your details are almost the same, except now you might want coverage for your new laptop. Additional coverage is only \$4 a month more for full coverage. Sound ok?

Sounds good!



## Autonomous Organization of Assistants

- Group of AI assistants that know every customer personally
- Eventually run large parts of company operations—from lead generation over marketing, sales, HR, or finance



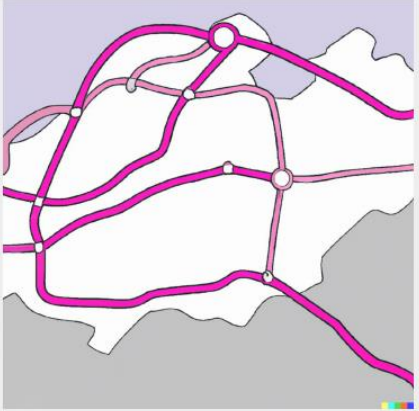
# Generative AI- PaintBox

What do you want to create?

An imaginary subway map  
in a coastal city.

Image dimensions:  by  (Max 2048)

Generate



# Generative AI- CodeAid

```
1 def binary_search(arr, x, l, r):_
2     if r >= l:
3         mid = l + (r - l) // 2
4         if arr[mid] == x:
5             return mid
6         elif arr[mid] > x:
7             return binary_search(arr, x, l, mid - 1)
8         else:
9             return binary_search(arr, x, mid + 1, r)
10    else:
11        return -1
```

< 1/2 > Accept

Tab

# Real-life challenges in NLP tasks



- Deep learning methods are data-hungry
- >50K data items needed for training
- The distributions of the source and target data must be the same
- Labeled data in the target domain may be limited
- This problem is typically addressed with **transfer learning**

# References



CH-7 - Speech and Language Processing by Daniel Jurafsky

---

[https://www.youtube.com/watch?v=Btmsly0j\\_dY&t=5s](https://www.youtube.com/watch?v=Btmsly0j_dY&t=5s)